

What to build, in order

Tick these off one at a time. Do not skip ahead to full pieces until the scale loop feels trustworthy. ChatGPT is the teacher. MusAI's own pitch analysis is the ear.

Architecture rule. Record or import audio → measure pitch and notes in the browser (pitchy) → store a JSON session (expected notes, cents, score) → send that JSON to OpenAI for coaching and chat. Never send the raw recording to ChatGPT and ask it to “hear” intonation. That does not work well and it is expensive.

How to use this list

- Finish a task before starting the next numbered item in the same phase.
- Each task has a *Done when* line. If you cannot tick that, the task is not done.
- You already have tuner, single-note trainer, scale studio, and optional GPT tips/chat. This plan finishes that, then extends it to a full piece.

PHASE 0 Make scales a complete product

Goal: a violinist records a scale and always gets a fair readout, then a teacher they can talk to.

☐ 01 0.1 Confirm the loop on your machine

Put `OPENAI_API_KEY` in `.env.local`, restart `npm run dev`, open Chrome at `http://127.0.0.1:3000`. Record a real C major scale in Scale studio (use Chrome, not Cursor's browser, so the mic works). Check results: coloured notes, score, teacher tip, and that chat answers you.
Done when: one real take produces results + a GPT tip (not only the template). If the tip says source is template, the key is missing or the API failed – fix that first.

☐ 02 0.2 Keep a small set of known recordings

Save 4-6 takes you understand: clean C major, skipped note, rushed, 2 octaves, out of tune on one string. Put them in the existing `musai audio/ fixtures` folder. Run them through Scale studio via Import.
Done when: you can say in one sentence what MusAI should conclude for each file, and Import + Analyse matches that most of the time.

☐ 03 0.3 Make scale detection trustworthy

Play those fixtures. If MusAI guesses the wrong tonic or octave span, improve `src/lib/detectScale.ts` or the confirm-scale UI. The player must always see what scale was used (“Heard from your take · C major, 1 octave”) and be able to pick another.
Done when: clean C major is detected as C major without you opening the guide; messy takes either detect correctly or ask you to confirm – they never fail silently.

☐ **04 0.4 Freeze the “session” as the source of truth**

Treat `ScalePracticeSessionV1` as the contract: expected notes, detected notes, cents, score, trend. Add a field if you must (for example `scaleSource` is already there). Do not let the UI invent notes the session does not contain.

Done when: results, coaching API, and chat all read from the same session object. One print of the JSON is enough to understand the take.

☐ **05 0.5 Save the last few takes on the device**

Store recent scale sessions in `localStorage` (no accounts yet). Let the player open “last take” and compare, even a simple list is enough.

Done when: you can close the tab, reopen Scale studio, and still open yesterday’s session.

PHASE 1 A teacher you can trust (still scales)

Goal: GPT always talks about *this* take. If the key is missing, templates still work.

☐ **01 1.1 Lock the coaching API**

Keep `/api/scale-coaching` as: session JSON in, `{ tip, trendLine, source }` out. Add tests that a fake session with a sharp E never produces a tip about a B that was not played.

Done when: unit tests fail if the model (or your parser) invents notes. Template fallback still works with no API key.

☐ **02 1.2 Lock the chat API**

Keep `/api/scale-coach-chat` grounded in measured notes. Chat must answer the student’s actual question, then drill from the data. Ban “I listened to your recording.”

Done when: you can ask “how do I fix the F?” and the reply names that F and a drill; asking “how’s your day?” does not dump the template tip.

☐ **03 1.3 Add a practice-plan object**

After each take, store 2-3 concrete drills (note, how many bows, what to listen for). Show them on results. Feed them into the next chat so GPT can say “repeat yesterday’s E-string drill.”

Done when: results always show a short plan, and chat can refer to it without inventing new notes.

☐ **04 1.4 Cap cost and hide secrets**

API key stays server-side only (already true). Set a model (e.g. `gpt-4o-mini`), max tokens, and a simple per-session message limit so a long chat cannot run away.

Done when: you can practise for 30 minutes without a surprise OpenAI bill, and the key never appears in client code.

PHASE 2 First full piece

Goal: the same loop, but the target is a score (MusicXML or MIDI), not a guessed scale.

☐ **01 2.1 Pick one short piece**

One phrase, 8–16 bars, violin only, slow. Get it as MusicXML or MIDI. This is your “C major” for pieces. Do not start from a PDF photo.
Done when: you have one file in the repo and can list the expected pitches in order (even in a unit test).

☐ **02 2.2 Build expected notes from the score**

Parse MusicXML/MIDI into the same shape you already use for scales: a list of expected MIDI pitches (and later, bar numbers and durations). Reuse `formatNoteLabel` and the green/yellow/red colour system. Do not invent a new visual language.
Done when: loading the piece shows the expected notes on a staff, with no recording yet.

☐ **03 2.3 Align a recording to those notes**

Reuse `collectPitchFrames` / stable runs. Map heard pitches onto the expected sequence (start with equal windows + octave wrap, same as scales). Mark skip / extra / unclear.
Done when: you play the phrase in time, and most notes light green/yellow/red on the staff instead of one blob score.

☐ **04 2.4 Same session JSON, same teacher**

Create `PiecePracticeSession` that looks like the scale session (notes, cents, score, trend) plus `pieceId` and bar numbers. Point coaching and chat at this JSON. Prompts say “this phrase,” not “this scale.”
Done when: after a take, GPT can say “bar 3, the F# is low” using only the session data.

☐ **05 2.5 Only then: longer music**

Whole movement, then rhythm (early/late), then camera/PDF scores. Each is a new project. Do not combine them.
Done when: the short piece works for you three days in a row before you add a second piece.

PHASE 3 Product around it (later)

Do this after a few other people can complete a scale and a short piece without you in the room.

☐ **01 3.1 Accounts and cloud saves**

Sign-in and store takes so GPT can compare this week to last week. Until then, device storage is enough.

☐ **02 3.2 Billing**

You pay OpenAI per take. Wrap that in a subscription only when strangers are using it.

☐ **03 3.3 Teacher / student**

Last. The measurement + coach loop has to be boringly reliable first.

Do not do yet

- Sending the .m4a/.wav into ChatGPT as the main feature.
- A new colour system, a new recorder, or a redesign of home.

- Rhythm, shifting drills, and a library of concertos before Phase 0 and 1 are ticked.
- Accounts, social, or “AI that listens like a professor.”

Suggested weekly rhythm

- **This week:** tasks 0.1–0.3 (real takes, fixtures, detection).
- **Next week:** 0.4–0.5 and 1.1–1.2 (session contract + GPT tests).
- **After that:** 1.3–1.4, then freeze scales and start 2.1 with one piece only.

Code you already have: Scale studio (`/practice/scale`), session type in `src/lib/scalePracticeTypes.ts`, coaching at `/api/scale-coaching`, chat at `/api/scale-coach-chat`, pitch in `src/lib/analyzePitch.ts`. The work is to finish that loop, then point it at a score instead of a scale.